Omówienie tematów numer 6, 18, 30 z egzaminu inżynierskiego WFiIS UŁ

Patryk Kantorski

UNIWERSYTET ŁÓDZKI

Kod CRC

Kod CRC (Cyclic Redundancy Check, pol. Cykliczny kod nadmiarowy) - system sum kontrolnych wykorzystywany do wykrywania przypadkowych błędów pojawiających się podczas przesyłania i magazynowania danych binarnych.

CRC jest jedną z najczęściej stosowanych metod kontroli poprawności przesyłanych danych. Mechanizm ten pozwala na wykrywanie błędów wynikających z zakłóceń w transmisji lub uszkodzenia danych w nośniku pamięci. CRC jest szeroko wykorzystywany w telekomunikacji, sieciach komputerowych, systemach wbudowanych i nośnikach danych, takich jak dyski twarde i pamięci flash.

Algorytm działania kodu CRC wykorzystuje trzy elementy:

- Binarny ciąg danych wejściowych (o długości N)
- Wielomian generacyjny (o długości r)
- Suma kontrolna (o długości r-1)

Wielomian generacyjny (lub wielomian generatorowy) w kodzie CRC (Cyclic Redundancy Check) to kluczowy element określający właściwości wykrywania błędów w przesłanych danych. Jest on używany do generowania sumy kontrolnej CRC, która jest dołączana do wiadomości przed transmisją.

Wielomian generacyjny G(x) to binarny wielomian (zwykle reprezentowany jako liczba szesnastkowa lub binarna), który definiuje algorytm obliczania CRC.

Przykłady popularnych wielomianów:

- **CRC-1**: $x+1$
  (wykorzystywany przy kontroli parzystości)
- **CRC-8**: $x^8 + x^7 + x^6 + x^4 + x^2 + 1$
  (standard DVB-S2 wykorzystywany w transmisji telewizji cyfrowej)
- **CRC-32**: $x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$
  (Ethernet, Wi-Fi, magistrale Serial ATA)

Suma kontrolna CRC (*Cyclic Redundancy Check*) to dodatkowa wartość obliczana na podstawie danych, służąca do wykrywania błędów podczas przesyłania lub przechowywania informacji. Jest to krótki kod nadmiarowy, dołączany do oryginalnych danych, który pozwala odbiorcy sprawdzić, czy dane nie zostały uszkodzone (np. przez zakłócenia w transmisji).

Algorytm generowania sumy kontrolnej CRC

1. Wiadomość traktowana jest jako wielomian M(x) (np. ciąg bitów 1010 odpowiada x^3+x).
2. Do wiadomości dodaje się r zer (gdzie r to stopień G(x)), tworząc $M'(x)=M(x) \cdot x^r$
3. Dzieli się M′(x) przez G(x) (operacja modulo-2, czyli XOR).
4. Reszta z dzielenia staje się sumą kontrolną CRC(x), która jest dołączana do oryginalnej wiadomości.

Algorytm weryfikacji

1. Otrzymana wiadomość ma postać $T(x) = M(x) \cdot x^r + CRC(x)$.
2. Dzieli się T(x) przez ten sam generator G(x) (operacja modulo-2, czyli XOR).
3. Sprawdzenie reszty
   - Jeśli reszta z dzielenia wynosi 0, wiadomość jest poprawna.
   - Jeśli reszta jest niezerowa, wiadomość została zakłócona.

Przedstawienie algorytmu generowania sumy kontrolnej CRC w opisie słownym

Wejście: dane_bin (ciąg bitów), wielomian_bin (wielomian generacyjny)
Wyjście: suma_kontrolna CRC

1. Dodaj zera do danych na końcu (liczba zer = długość wielomianu - 1)
2. Ustaw bufor na pierwsze bity danych o długości wielomianu
3. Powtarzaj do przetworzenia wszystkich bitów danych:
   a. Jeśli pierwszy bit bufora to 1:
      - XOR bufor z wielomianem generacyjnym
   b. Przesuń bufor w prawo i dodaj kolejny bit danych
4. Zwróć resztę w buforze jako kod CRC

Przykład

Niech dana będzie czternastobitowa wiadomość postaci: 11010011101110, którą chcemy dzielić przez wybrany przez nas czterobitowy wielomian x^3+x+1. Do wiadomości dodajemy przyrostek trzech zer, na którym zostanie zapisana suma kontrolna CRC.

```
110 10011101110|000
10 1 1
011 00011101110|000
  10 11
001 11011101110|000
    1 011
000 10111101110|000
       1011
000 00001101110|000
```

```
            1011
000 00000110110|000
             1011
000 00000011010|000
              1011
000 00000001100|000
               1011
000 00000000111|000
                101 1
000 00000000010|100
                 10 11
000 00000000000|010
```

Otrzymaliśmy sumę kontrolną CRC o wartości $010_2 = 2_{10}$.

Spróbujmy zweryfikować naszą transmisję. W tym celu do sygnału dodajemy przyrostek w postaci wartości sumy kontrolnej CRC oraz dzielimy dane wejściowe przez ten sam wielomian.

```
1101001110111 0|010
1011
0110001110111 0|010
  1011
```

[…]

```
000000000000111|010
                101 1
000000000000010|110
                  10 11
000000000000000|000
```

Wynikiem operacji jest 0, więc nie znaleziono zakłócenia w transmisji danych.

Przykład oszukania sumy kontrolnej CRC

Jeśli znamy oryginalną sumę CRC i chcemy zmodyfikować dane, ale zachować poprawną wartość CRC, możemy wykorzystać właściwość liniowości CRC (w przypadku niektórych trybów).

Ponieważ niektóre CRC (np. podstawowe CRC-32) są addytywne w arytmetyce modulo-2, możemy:

- Zmienić dane w przewidywalny sposób i obliczyć jak zmieni się CRC.
- Dobrać zmianę danych, aby CRC pozostało takie samo.

Przykład:

- Załóżmy, że $CRC(D) = C$.
- Jeśli zmodyfikujemy dane D przez XOR z pewnym wzorcem Δ, nowe CRC będzie:
- $CRC(D \oplus \Delta) = CRC(D) \oplus CRC(\Delta) = C \oplus CRC(\Delta)$
- Aby CRC się nie zmieniło, musimy wybrać Δ takie, że $CRC(\Delta) = 0$.

Zadziała to tylko dla CRC bez nieliniowych operacji (np. bez finalnego XOR lub odwracania bitów) i dodatkowych zabezpieczeń.

Procesy, wątki oraz algorytmy ich planowania

Procesy

Proces - załadowany do pamięci komputera program, który posiada własne zasoby i kontekst wykonania. Strukturą zawierającą dane o procesie jest PCB (Process Control Block).

W skład obiektu PCB wchodzi:

- identyfikator (PID) – unikalny, jednoznacznie identyfikujący numer procesu
- kod programu – instrukcje wykonywane przez procesor
- licznik rozkazów – rejestr procesora, który przechowuje adres następnej instrukcji do wykonania
- stos programu - używany do przechowywania lokalnych zmiennych, adresów powrotu i danych funkcji.
- sekcja danych - obszar pamięci procesu przeznaczony na przechowywanie zmiennych globalnych i statycznych, które są inicjalizowane przez programistę

Wykonywanie procesów jest nadzorowane przez jądro systemu, a sposób ich zarządzania zależy od konkretnej platformy. Procesy są zorganizowane w strukturę hierarchiczną – każdy z nich posiada proces nadrzędny (macierzysty) i może inicjować procesy potomne za pomocą odpowiednich wywołań systemowych, tworząc w ten sposób układ przypominający drzewo.

Dodatkowo system operacyjny przydziela każdemu procesowi określone zasoby, takie jak:

- czas procesora
- pamięć
- dostęp do urządzeń wejścia-wyjścia
- pliki

Pięciostanowy model procesu

Proces może znajdować się w jednym z kilku stanów:

- Nowy (New) – proces został utworzony, ale jeszcze nie jest wykonywany.
- Gotowy (Ready) – proces czeka na przydział procesora.
- Wykonywany (Running) – proces aktualnie jest wykonywany przez procesor.
- Oczekujący (Waiting/Blocked) – proces czeka na zasób (np. dane z dysku, wejście/wyjście).
- Zakończony (Terminated) – proces zakończył działanie i zwolnił zasoby.

Procesy mogą być:

- Jednowątkowe (single-threaded) – mają tylko jeden wątek wykonania.
- Wielowątkowe (multi-threaded) – mogą uruchamiać wiele wątków w obrębie jednego procesu.

Procesy komunikują się ze sobą za pomocą mechanizmów IPC (Inter-Process Communication), takich jak kolejki komunikatów, pamięć współdzielona czy potoki (pipes).

Wątki

Wątek (ang. thread) – najmniejsza jednostka wykonawcza w systemie operacyjnym, która może być niezależnie planowana przez scheduler (planistę). Każdy proces składa się z co najmniej jednego wątku, ale może zawierać ich wiele, co pozwala na równoczesne wykonywanie różnych zadań.

Wątki należące do tego samego procesu współdzielą jego zasoby (np. pamięć, otwarte pliki), ale każdy wątek ma własny:

- licznik rozkazów
- stos
- rejestry procesora

Różnica między procesem a wątkiem

Cecha	Proces	Wątek
Definicja	Samodzielna jednostka wykonawcza programu.	Lekka jednostka wykonawcza w obrębie procesu.
Pamięć	Każdy proces ma własną przestrzeń adresową.	Wszystkie wątki jednego procesu współdzielą pamięć.
Zasoby	Posiada własne zmienne, pliki i uchwyty.	Wątki korzystają z tych samych zasobów procesu.
Komunikacja	Wymaga mechanizmów IPC (kolejki, potoki).	Może odbywać się bezpośrednio przez współdzieloną pamięć.
Koszt przełączenia	Wysoki – przełączanie wymaga zmiany kontekstu procesora.	Niski – przełączanie między wątkami jest szybsze.

Typy wątków

a) Wątki użytkownika (User Threads)

- Tworzone i zarządzane na poziomie aplikacji, bez bezpośredniego wsparcia jądra systemu.
- Nie są widoczne dla systemu operacyjnego, a przełączanie między nimi odbywa się w przestrzeni użytkownika.
- Wadą jest brak możliwości rzeczywistego wykonywania wielu wątków jednocześnie na różnych rdzeniach procesora.
- Przykłady: biblioteki wielowątkowe (np. POSIX Threads – pthreads dla języka C).

b) Wątki jądra (Kernel Threads)

- Obsługiwane przez system operacyjny i zarządzane na poziomie jądra.
- Mogą być wykonywane równolegle na wielu rdzeniach procesora.
- Ich obsługa jest bardziej kosztowna niż wątków użytkownika.
- Przykłady: wątki w systemach Windows (CreateThread), Linux (clone, pthreads z wsparciem jądra).

c) Model hybrydowy (Hybrid Threads)

- Łączy zalety wątków użytkownika i jądra.
- System zarządza wątkami jądra, a użytkownik może tworzyć własne wątki użytkownika.

Zalety wielowątkowości

- Efektywne wykorzystanie CPU – wątki mogą działać równolegle na różnych rdzeniach procesora.
- Szybsza komunikacja – wątki dzielą pamięć, co eliminuje konieczność stosowania kosztownych mechanizmów IPC.
- Lepsza responsywność aplikacji – np. w przeglądarce internetowej interfejs działa w jednym wątku, a ładowanie strony w innym.
- Wydajność w serwerach – np. serwery sieciowe obsługujące wielu użytkowników mogą przydzielać osobne wątki do obsługi każdego połączenia.

Problemy związane z wielowątkowością

a) Synchronizacja wątków

Ponieważ wątki dzielą tę samą pamięć, mogą wystąpić problemy takie jak:

- Warunki wyścigu (Race Conditions) – dwa wątki próbują zmodyfikować tę samą zmienną w tym samym czasie.
- Zakleszczenia (Deadlocks) – dwa wątki oczekują na zasoby zablokowane przez siebie nawzajem.
- Zagłodzenie (Starvation) – jeden wątek nie otrzymuje dostępu do zasobów, ponieważ inne wątki go blokują.

b) Mechanizmy synchronizacji

Aby uniknąć problemów, stosuje się mechanizmy synchronizacji:

- Muteksy (Mutex) – blokują zasób, dopóki jeden wątek go nie zwolni.
- Semafory – licznik pozwalający określonej liczbie wątków uzyskać dostęp do zasobu.
- Zmienna warunkowa (Condition Variable) – pozwala wątkom czekać na określone zdarzenia.
- Sekcje krytyczne – fragmenty kodu, które tylko jeden wątek może wykonywać jednocześnie.

Podstawowe algorytmy planowania CPU

a) FCFS (First-Come, First-Served)

- Procesy wykonują się w kolejności przybycia.
- Może powodować długi czas oczekiwania (problem konwoju).

b) SJF (Shortest Job First)

- Wybiera proces o najkrótszym czasie wykonania.
- Optymalny pod względem średniego czasu oczekiwania, ale może prowadzić do zagłodzenia długich procesów.

c) RR (Round Robin)

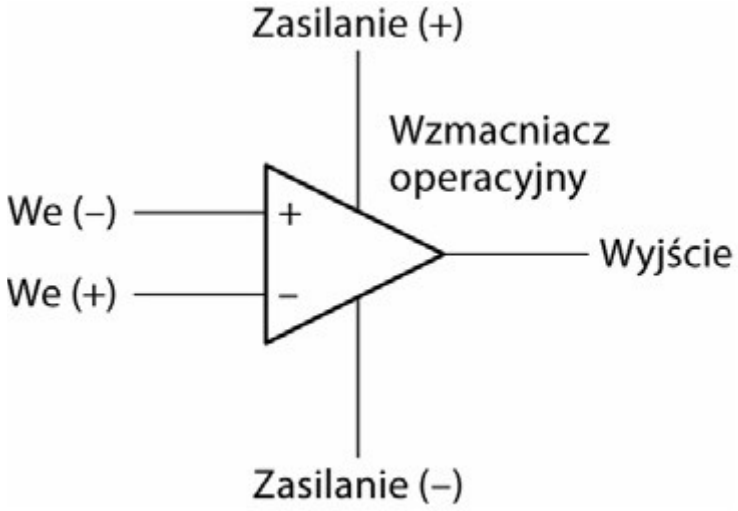
- Każdy proces dostaje określony kwant czasu, po czym przechodzi na koniec kolejki.
- Dobrze sprawdza się w systemach interaktywnych.

d) Priority Scheduling

- Procesy mają przypisane priorytety – najwyższy priorytet jest wykonywany jako pierwszy.
- Może powodować zagłodzenie procesów o niskim priorytecie.

Wzmacniacz operacyjny

Wzmacniacz operacyjny (ang. Operational Amplifier, Op-Amp) to uniwersalny, wysokowydajny wzmacniacz elektroniczny o bardzo dużym wzmocnieniu napięciowym, zaprojektowany do wykonywania operacji matematycznych (np. dodawania, całkowania) oraz przetwarzania sygnałów analogowych. Działa w układach z sprzężeniem zwrotnym, co pozwala na precyzyjne sterowanie jego charakterystyką.



Typowe zastosowania wzmacniaczy operacyjnych

1. Wzmacniacz odwracający:

Schemat i działanie:

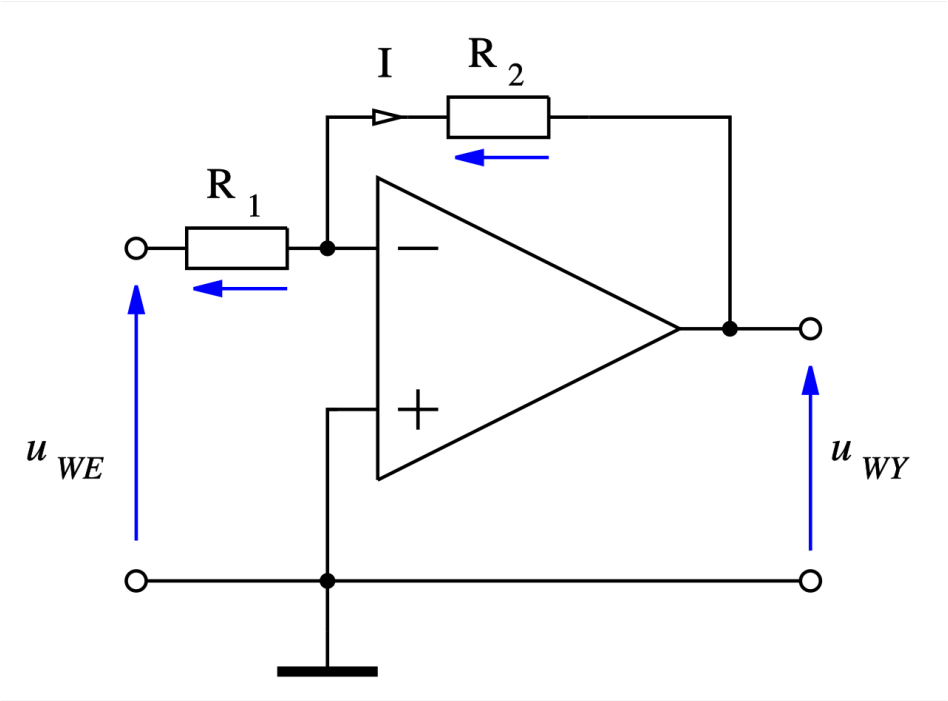
- Wejście sygnału podłączone do odwracającego wejścia (-) przez rezystor R1.
- Wejście nieodwracające (+) uziemione.
- Rezystor sprzężenia zwrotnego R2 łączy wyjście z wejściem odwracającym.

Zastosowania:

- Konwersja sygnału (zamiana fazy o 180°).
- Wzmacnianie sygnałów analogowych.

Typowe zastosowania wzmacniaczy operacyjnych

1. Wzmacniacz odwracający:



$$U_{wy} = -\frac{R_2}{R_1} U_{we}$$

Typowe zastosowania wzmacniaczy operacyjnych

2. Wzmacniacz nieodwracający:

Schemat i działanie:

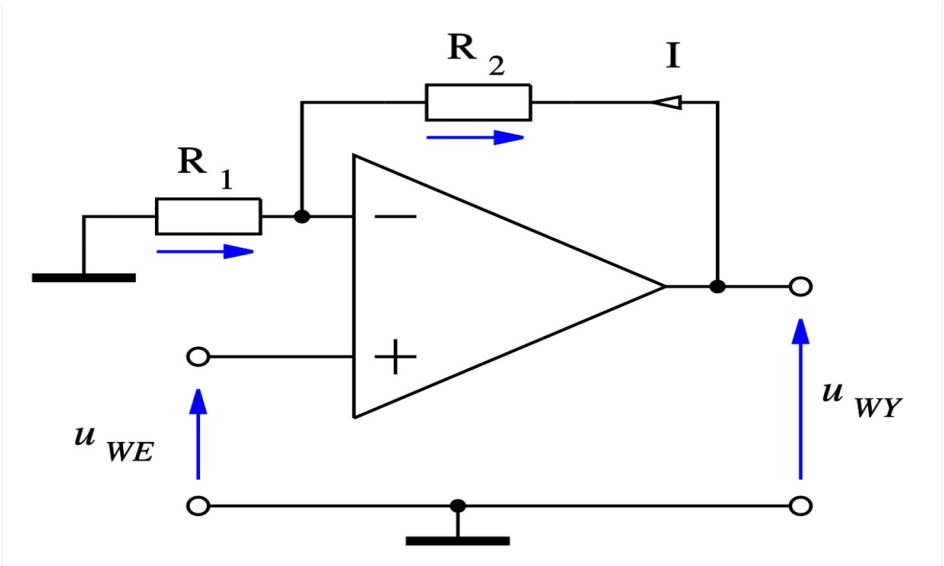
- Sygnał wejściowy podawany na wejście nieodwracające (+).
- Odwracające wejście (-) sprzężone przez dzielnik napięcia (R1 i R2).
- Wzmocnienie napięciowe:

Zastosowania:

- Wzmacnianie sygnałów bez zmiany fazy.
- Buforowanie sygnałów o wysokiej impedancji.

Typowe zastosowania wzmacniaczy operacyjnych

2. Wzmacniacz nieodwracający:



$$U_{wy} = \left(1 + \frac{R_2}{R_1}\right) U_{we}$$

Typowe zastosowania wzmacniaczy operacyjnych

3. Wzmacniacz różnicowy:

Schemat i działanie:

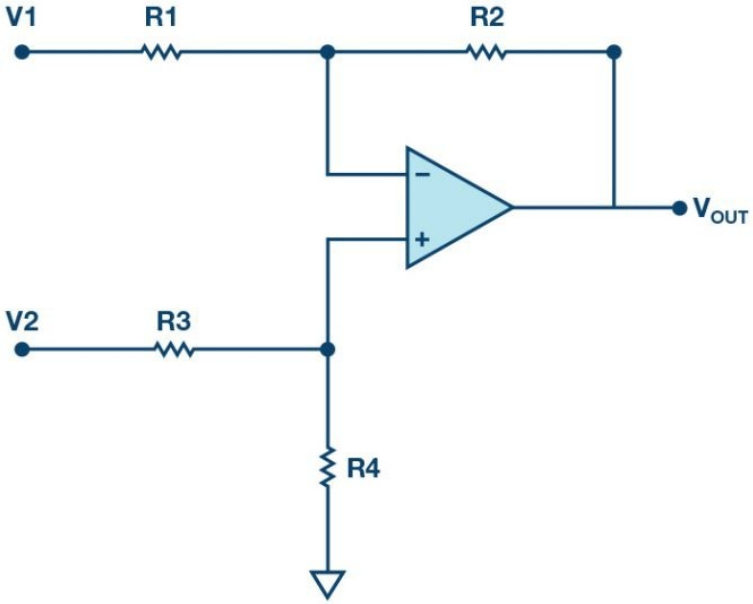
- Dwa rezystory wejściowe {R1, R3} i dwa rezystory sprzężenia zwrotnego {R2, R4}.
- Wzmacnia różnicę napięć między dwoma wejściami.

Zastosowania:

- Eliminacja zakłóceń w sygnale.
- Przetwarzanie sygnałów różnicowych np. w czujnikach pomiarowych.

Typowe zastosowania wzmacniaczy operacyjnych

3. Wzmacniacz różnicowy:



$$U_{wy} = \frac{R_2}{R_1} U_{we}$$

Typowe zastosowania wzmacniaczy operacyjnych

4. Wzmacniacz całkujący:

Schemat i działanie:

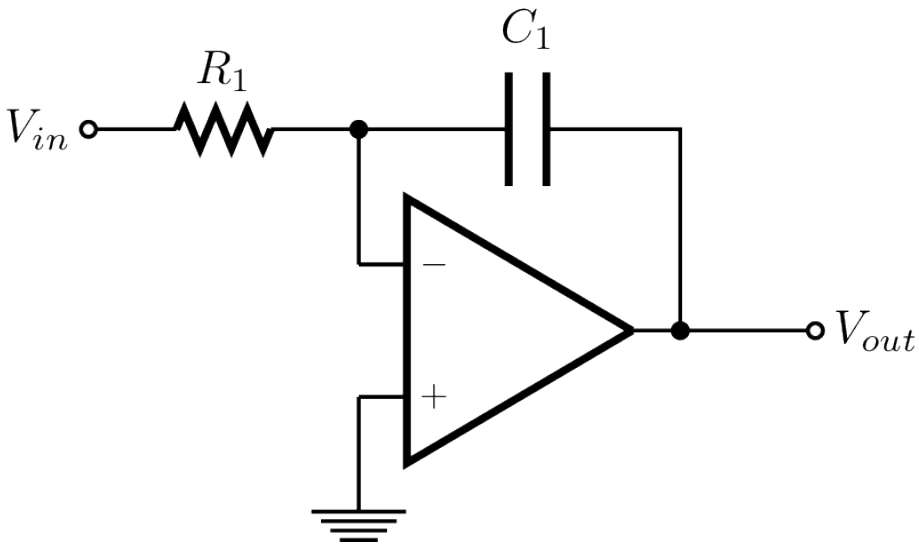
- Zastąpienie rezystora sprzężenia zwrotnego kondensatorem C.

Zastosowania:

- Filtry dolnoprzepustowe.
- Generatory rampowe.
- Integracja sygnałów w systemach pomiarowych.

Typowe zastosowania wzmacniaczy operacyjnych

4. Wzmacniacz całkujący:



$$U_{wy} = -\frac{1}{RC}\int U_{we}\,dt$$

Typowe zastosowania wzmacniaczy operacyjnych

5. Wzmacniacz różniczkujący:

Schemat i działanie:

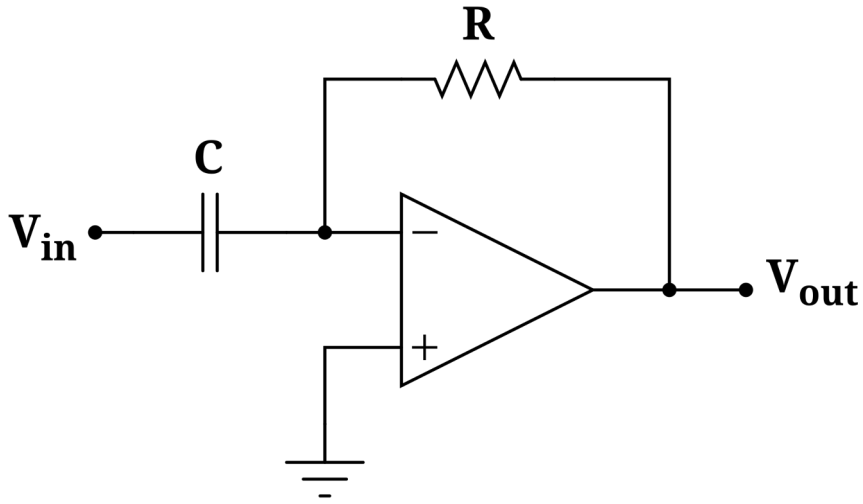
- Kondensator na wejściu zamiast rezystora.

Zastosowania:

- Filtry górnoprzepustowe.
- Wykrywanie gwałtownych zmian sygnału (np. krawędzie impulsów).

Typowe zastosowania wzmacniaczy operacyjnych

5. Wzmacniacz różniczkujący:



$$U_{wy} = -RC\frac{dU_{we}}{dt}$$

Porównanie parametrów idealnego WO z parametrami rzeczywistych WO

Parametr	Definicja	Co oznacza	Idealny WO	Rzeczywisty WO
Wzmocnienie napięciowe	Stosunek napięcia wyjściowego do napięcia wejściowego.	Określa, ile razy wzmacniacz powiększa sygnał wejściowy.	Nieskończone	Bardzo wysokie, ale skończone (np. 100 000 – 1 000 000)
Impedancja wejściowa	Opór, jaki wzmacniacz stawia sygnałowi wejściowemu.	Wysoka impedancja oznacza, że układ nie obciąża źródła sygnału.	Nieskończone	Bardzo duża, ale skończona (np. MΩ – GΩ)
Impedancja wyjściowa	Opór, jaki wzmacniacz stawia przepływowi prądu do obciążenia	Im niższa, tym stabilniejsze napięcie na wyjściu.	0Ω	Mała, ale niezerowa (np. kilka Ω)

Porównanie parametrów idealnego WO z parametrami rzeczywistych WO

Parametr	Definicja	Co oznacza	Idealny WO	Rzeczywisty WO
Pasmo przenoszenia	Zakres częstotliwości, w którym wzmacniacz może pracować efektywnie.	Im szersze pasmo, tym lepiej wzmacniacz może przetwarzać szybkie zmiany sygnału.	Nieskończone	Ograniczone, zależne od wzmocnienia
Napięcie offsetu	Różnica napięć na wejściach wzmacniacza	Powoduje niezerowe napięcie na wyjściu, nawet gdy sygnał wejściowy wynosi 0V.	0V	Kilka mV do µV

Porównanie parametrów idealnego WO z parametrami rzeczywistych WO

Parametr	Definicja	Co oznacza	Idealny WO	Rzeczywisty WO
Prąd polaryzacji	Niewielki prąd stały, który przepływa przez wejścia wzmacniacza do wewnętrznych tranzystorów lub stopni wejściowych.	Powoduje niepożądane napięcie offsetowe.	0A	Kilka nA do pA
Dryft temperaturowy	Zmiany parametrów wzmacniacza w zależności od temperatury	Może wpływać na stabilność działania układu.	Brak	Występuje, zależy od modelu

Bibliografia

https://www.cl.cam.ac.uk/research/srg/projects/fairisle/bluebook/21/crc/crc.html
https://ucgosu.pl/2017/01/jak-dziala-crc/

https://www.ibm.com/docs/pl/aix/7.3?topic=performance-processes-threads
https://web.archive.org/web/20190517184831/http://vlsi.imio.pw.edu.pl/pmk/pmk_W11_1.html

Dziękuję za uwagę